

PYTHON PROGRAMMING FOR COMPUTING

Unit - 1

Dr. Susheela Dahiya

Associate Professor

Department of Computer Science & Engineering

Graphic Era Hill University

Dehradun

Python

- Python is a widely used general-purpose, high level programming language.
- *Python* was conceived in the late **1980s** and was named after the *BBC TV show Monty Python's Flying Circus*.
- Guido van Rossum started implementing **Python at CWI (Centrum Wiskunde & Informatica) in the Netherlands in December of 1989**.
- This was a successor to the ABC programming language which was capable of **exception handling and interfacing** with the Amoeba operating system.
- Python was first released on February 20, 1991.
- Python is developed by Python Software Foundation.
- On **October 16 of 2000, Python 2.0 released** with many new features.
- Then **Python 3.0 was released on December 3, 2008**.



Introduction

- Python is a high-level language.
- Python is a dynamically typed language.
 - The type of the variable is determined only during runtime.
- Python is a case sensitive language.
 - Differentiates between lowercase and uppercase letters.
- Python is an interpreted language.
- Python is an interactive and object-oriented programming language.
- Python is portable: it runs on many Unix variants including Linux and macOS, and on Windows.
- The Python IDLE (Integrated Development Environment) executes instructions one line at a time.

Features of Python

- **High-level language**
- **Dynamically typed language**
 - The type of the variable is determined only during runtime.
- **Case sensitive language**
 - Differentiates between lowercase and uppercase letters.
- **Easy**
 - Python is very easy to learn and understand; any beginner can learn Python easily.
- **Interpreted**
 - The Python IDLE (Integrated Development Environment) executes instructions one line at a time. This makes it easy to test and debug.
- **Object-Oriented**
 - The Python programming language supports classes and objects and hence it is object-oriented.
- **Free and Open Source**
 - The language and its source code are available to the public for free; there is no need to buy a costly license.
- **Portable**
 - Since Python is open-source, you can run it on Windows, Mac, Linux or any other platform. Your programs will work without any need to change it for every machine.
- **GUI Programming**
 - You can use it to develop a GUI (Graphical User Interface). One way to do this is through **Tkinter**.
- **Large Python Library**
 - Python provides you with a large standard library.
- **Data Science** - 'Numpy' and 'Pandas'.

Why learn Python?

- Fun-to-use "Scripting language"
- Object-oriented
 - Highly educational
- Very easy to learn
- Powerful, scalable, easy to maintain
 - high productivity
 - Lots of libraries
- Glue language
 - Python is often described as a "glue language," meaning it can let disparate code (typically libraries with C language interfaces) interoperate.
 - Interactive front-end for C/C++ code.
 - If you have applications or program domains that you would like to hitch up, but cannot talk to each other directly, you can use Python to connect them.
- Reduce development time
- Reduce code length
- Easy to learn and use as developers
- Easy to understand codes
- Easy to do team projects
- Easy to extend to other languages

Where to use Python?

- System management (i.e., scripting)
- Graphic User Interface (GUI)
- Internet programming
- Database (DB) programming
- Text data processing
- Distributed processing
- Numerical operations
- Graphics
- And so on...

Who uses Python today?

Python is being applied in real revenue-generating products by real companies as follows:

- Google makes extensive use of Python in its web search system and employs Python's creators.
- Intel, Cisco, Hewlett-Packard, Seagate, Qualcomm, and IBM uses Python for hardware testing.
- ESRI uses Python as an end-user customization tool for its popular GIS mapping products.
- The YouTube video sharing service is largely written in Python

Timeline

- Python born, name picked - Dec 1989
 - By Guido van Rossum, now at GOOGLE
- First public release (USENET) - Feb 1991
- python.org website - 1996 or 1997
- 2.0 released - 2000
- Python Software Foundation - 2001
- ...
- 3.0 released – December 3, 2008
- Current version: 3.11.x

The Python interpreter and the extensive standard library are freely available in source or binary form for all major platforms from the Python Web site, <https://www.python.org/>

Python version numbering scheme

- Python versions are numbered A.B.C or A.B. (Ex: Python 3.11.4)
- A is the major version number – it is only incremented for really major changes in the language.
- B is the minor version number, incremented for less earth-shattering changes.
- C is the micro-level – it is incremented for each bugfix release.

Command Line

- python

```
Last login: Wed Sep 20 11:11:54 on ttys000
(base) susheela@Susheelas-MacBook-Pro ~ % python
Python 3.7.6 (default, Jan  8 2020, 13:42:34)
[Clang 4.0.1 (tags/RELEASE_401/final)] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> exit()
(base) susheela@Susheelas-MacBook-Pro ~ %
```

- python **--help**

- Print a short description of all command line options and corresponding environment variables and exit.

- python **--version**

- Print the Python version number and exit.

- python myscript.py

- Used for invocation / execution of a script

Installing Python

- You can install either Interactive or Script-based IDE for writing Python program
- Interactive -
 - You type directly to Python one line at a time, and it responds.
 - Ex – Jupyter Notebook
- Script
 - You enter a sequence of statements (lines) into a file using a text editor and tell Python to execute the statements in the file
 - Ex - PyCharm

Installing Jupyter Notebook

- Step 1: Install the latest Python version from the Python Web site, <https://www.python.org/>
- Step 2: Install JupyterLab with pip:
 - `pip install jupyterlab`
- Step 3: Launch JupyterLab with:
 - `jupyter lab`
- Step 4: Install the classic Jupyter Notebook with:
 - `pip install notebook`
- Step 5: To run the notebook:
 - `jupyter notebook`

Installing PyCharm

- Step 1: Download and install the latest Python version from the Python Web site, <https://www.python.org/>
- Step 2: Download PyCharm Community Edition from the JetBrains web site, <https://www.jetbrains.com/pycharm/>
- Step 3: Install PyCharm by following the installation steps
- Step 4: Create a new Project
- Step 5: Create python files inside your project for writing python scripts

Using Python as a Calculator

- The Python interpreter acts as a simple calculator: you can type an expression at it and it will write the value. Expression syntax is straightforward: the operators `+`, `-`, `*` and `/` work just like in most other languages; parentheses `()` can be used for grouping. For example:
- The integer numbers (e.g. 2, 4, 20) have type `int`, the ones with a fractional part (e.g. 5.0, 1.6) have type `float`.
- Division (`/`) always returns a float. To do **floor division** and get an integer result (discarding any fractional result) you can use the `//` operator; to calculate the remainder you can use `%`:
- ```
>>> 2 + 2
4
```
- ```
>>> 50 - 5*6
20
```
- ```
>>> 8 / 5
1.6
```
- ```
>>> 8 // 5
1
```
- ```
>>> (50 - 5*6) / 4
5.0
```

# Comments

---

- Single Line Comments
  - # This is a comments.
- Multi-line Comments - Not supported
  - Triple-quoted string is also ignored by Python interpreter and can be used as a multiline comments:  
    `""" This  
    is  
    a  
    multi-line  
    comment"""`

# The print() Function

---

- The print() function prints the specified message to the screen, or any other standard output device.
- The message can be a string, or any other object, the object will be converted into a string before written to the screen.

## Syntax

```
print(object(s), sep=separator, end=end, file=file, flush=flush)
```

## Parameter Values

| Parameter              | Description                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------------------|
| <i>object(s)</i>       | Any object, and as many as you like. Will be converted to string before printed                       |
| <i>sep='separator'</i> | Optional. Specify how to separate the objects, if there is more than one. Default is ' '              |
| <i>end='end'</i>       | Optional. Specify what to print at the end. Default is '\n' (line feed)                               |
| <i>file</i>            | Optional. An object with a write method. Default is sys.stdout                                        |
| <i>flush</i>           | Optional. A Boolean, specifying if the output is flushed (True) or buffered (False). Default is False |

# The print() Function

---

In [8]: `print("Python Programming")`

Python Programming

In [9]: `print("Python")`  
`print("Programming")`

Python  
Programming

In [12]: `print("Python\nProgramming")`

Python  
Programming

In [15]: `print("Python\tProgramming")`

Python   Programming

---

In [17]: `print('Python', 'Programming', sep='-')`

Python-Programming

In [10]: `print("Python", end=" ")`  
`print("Programming")`

Python Programming

In [11]: `print("Python", end="\t")`  
`print("Programming")`

Python   Programming

In [65]: `print("Python\\nProgramming")`

Python\nProgramming

In [66]: `print("This is \"Python\" Class")`

This is "Python" Class

In [67]: `print("This is \'Python\' Class")`

This is 'Python' Class

# Variables

- A name associated with an object
- Assignment used for binding

$m = 64$

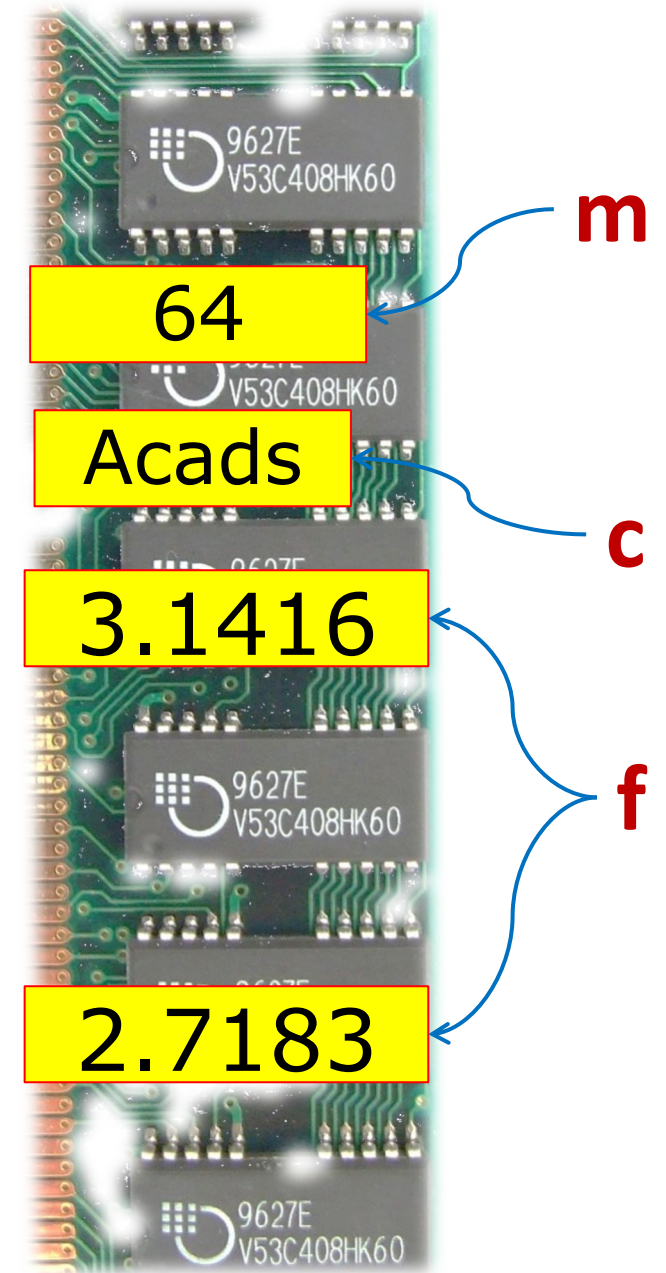
$c = \text{'Acads'}$

$f = 3.1416$

- Variables can change their bindings

$f = 2.7183$

- Dynamic Types
  - The data type of the variable is determined only during runtime.
- You cannot use reserved words as variable names / identifiers



# Assignment Statement

---

- A simple assignment statement

*Variable = Expression;*

- Computes the value (object) of the expression on the right-hand side expression (**RHS**)
- Associates the name (variable) on the left-hand side (**LHS**) with the RHS value
- **=** is known as the assignment operator.

# Multiple Assignments

---

- Python allows multiple assignments

**x, y = 10, 20**

Binds x to 10 and y to 20

- Evaluation of multiple assignment statement:
  - All the expressions on the RHS of the **=** are first evaluated **before any binding happens**.
  - Values of the expressions are bound to the corresponding variable on the LHS.

**x, y = 10, 20**  
**x, y = y+1, x+1**

x is bound to 21 and y to 11 at the end of the line

# Naming Conventions for Variables

---

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and \_ )
- Variable names are case-sensitive (name, Name and NAME are three different variables)

# Naming Conventions for Variables

---

```
name = "Susheela"
print(name)
```

Susheela

```
name_1 = "Susheela"
print(name_1)
```

Susheela

```
_name1 = "Susheela"
print(_name1)
```

Susheela

```
1_name = "Susheela"
print(1_name)
```

```
File "<ipython-input-6-28715c56ddce>", line 1
```

```
 1_name = "Susheela"
```

```
 ^
```

```
SyntaxError: invalid token
```

# Python Reserved Words

---

- `import` keyword  
`print(keyword.kwlist)`

- Output -

['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

- You cannot use reserved words as variable names / identifiers

# id() function

- The id() function is a built-in function that returns the unique identifier (id) of an object (any object, variable, String, List, Class etc.)
- The id is assigned to the object when it is created.
- The identifier is an integer, which represents the memory address of the object and will be different for each time you run the program. (except for some object that has a constant unique id)
- The id() function is commonly used to check if two variables or objects refer to the same memory location.
- All objects in Python has its own unique id.

**Syntax:** id(object)

**Return:** a unique integer for a given object

```
number = 15
print(id(number))
```

```
number = 11
print(id(number))
```

```
number = 15.3
print(id(number))
```

```
number = 15.3
print(id(number))
```

4557624896

4557624768

140212070352304

140212070353552

# The format function

---

Syntax: { }.format(value1, value2, ...)

Parameters:

- value1, value2, ... : Required,
  - One or more values that should be formatted and inserted in the string.
  - Can be of any data type (an integer, floating point numeric constant, string, characters) or even variables.

Return type: Returns a formatted string with the value passed as parameter in the placeholder position.

```
In [22]: var_1 = "Python"
var_2 = "Java"
print("I like {} and {} programming languages".format(var_1, var_2))
```

I like Python and Java programming languages

```
In [23]: var_1 = "Python"
var_2 = "Java"
print("I like {1} and {0} programming languages".format(var_1, var_2))
```

I like Java and Python programming languages

```
In [21]: message = "For only {price:.2f} dollars!"
print(message.format(price = 49))
```

For only 49.00 dollars!

```
In [29]: value = 123.456789
print("{:.2f}".format(value))
```

123.46

```
In [57]: print("{:8.2f}".format(12.45678))
print("{:8.2f}".format(212.45678))
print("{:8.2f}".format(1212.45678))
```

12.46  
212.46  
1212.46

# Formatted string literals

---

Formatted string literals begin a string with `f` or `F` before the opening quotation mark or triple quotation mark. Inside this string, you can write a Python expression between `{` and `}` characters that can refer to variables or literal values.

```
In [38]: name = "Susheela"
 message = f"Hello {name}"
 print(message)

Hello Susheela
```

```
In [39]: name = "Susheela"
 message = F"Hello {name}"
 print(message)

Hello Susheela
```

# Main Python Data Types

---

Every value in Python is called an “**object**”. And every object has a specific data type. The three most-used data types are as follows:

- **Integers (int)** — An integer is a positive or negative number without floating point. Ex: -2, -1, 0, 1, 2, 3, 4, 5
- **Floating-point numbers (float)** — A float is a positive or negative number with floating point precision. Ex: -1.25, -1.0, --0.5, 0.0, 0.5, 1.0, 1.25
- **Strings** — Python Strings are sequences of characters. For example, the word “hello”.
  - In Python 3, They are immutable which means that you can not alter the characters without creating a new string. Ex: **‘Hello!’**, **‘How are you?’**
- The three main ways to create strings are the following.
  - 1. Single quotes - 'Yes'
  - 2. Double quotes - "Yes"
  - 3. Triple quotes (multi-line) - 

```
"""Yes
We Can"""
```
- **Boolean** — The Boolean data type is a truth value, either **True** or **False**.

# type() function

- Built-in function that is used to return the type of data stored in the objects or variables in the program

```
number = 15
print(type(number))
```

```
<class 'int'>
```

```
number = 13.6
print(type(number))
```

```
<class 'float'>
```

```
number = 3 + 4j
print(type(number))
```

```
<class 'complex'>
```

```
name = "Susheela"
print(type(name))
```

```
<class 'str'>
```

```
name = 'Susheela'
print(type(name))
```

```
<class 'str'>
```

```
text = """This
is
Python
Programming
Class."""
print(type(text))
```

```
<class 'str'>
```

```
result = True
print(type(result))
```

```
<class 'bool'>
```

```
result = False
print(type(result))
```

```
<class 'bool'>
```

```
result = 'False'
print(type(result))
```

```
<class 'str'>
```

# Simple Input/output

---

- `input()` function is used to take input from the user at run time.
- Syntax – `input(prompt)`
- By default, the user input is String.
- The input can be converted into required input type by using the appropriate function
  - `int()` – used to convert the input into integer type
  - `float()` – used to convert the input into integer type
  - `bool()` – used to convert the input into boolean type

```
num_1 = input("Enter a number: ")
num_2 = input("Enter another number: ")
print("Sum = ", num_1+num_2)
```

Enter a number:

```
num_1 = input("Enter a number: ")
num_2 = input("Enter another number: ")
print("Sum = ", num_1+num_2)
```

Enter a number: 23

Enter another number:

```
num_1 = input("Enter a number: ")
num_2 = input("Enter another number: ")
print("Sum = ", num_1+num_2)
```

Enter a number: 23

Enter another number: 76

Sum = 2376

```
print(type(num_1))
print(type(num_2))
```

<class 'str'>

<class 'str'>

```
num_1 = int(input("Enter a number: "))
num_2 = int(input("Enter another number: "))
print("Sum = ", num_1+num_2)
```

```
Enter a number: 12
Enter another number: 34
Sum = 46
```

```
print(type(num_1))
print(type(num_2))
```

```
<class 'int'>
<class 'int'>
```

```
num_1 = int(input("Enter a number: "))
num_2 = float(input("Enter another number: "))
print("Sum = ", num_1+num_2)
```

```
Enter a number: 12
Enter another number: 34
Sum = 46.0
```

```
print(type(num_1))
print(type(num_2))
```

```
<class 'int'>
<class 'float'>
```

# Math/ Arithmetic Operators

| Operators       | Operation               | Example                    |
|-----------------|-------------------------|----------------------------|
| <code>**</code> | Exponent                | <code>2 ** 3 = 8</code>    |
| <code>%</code>  | Modulus/Remainder       | <code>22 % 8 = 6</code>    |
| <code>//</code> | Floor /Integer division | <code>22 // 8 = 2</code>   |
| <code>/</code>  | Division                | <code>22 / 8 = 2.75</code> |
| <code>*</code>  | Multiplication          | <code>3 * 3 = 9</code>     |
| <code>-</code>  | Subtraction             | <code>5 - 2 = 3</code>     |
| <code>+</code>  | Addition                | <code>2 + 2 = 4</code>     |

```
x, y = 3, 2
print(x + y) # 5
print(x - y) # 1
print(x * y) # 6
print(x / y) # 1.5
print(x // y) # = 1
print(x % y) # = 1
print(-x) # = -3
print(abs(-x)) # = 3
print(int(3.9)) # = 3
print(float(3)) # = 3.0
print(x ** y) # = 9
```

# Relational / Comparison Operators

- Used to compare two values
- Returns Boolean value

| Operator | Name                     | Example |
|----------|--------------------------|---------|
| ==       | Equal                    | x == y  |
| !=       | Not equal                | x != y  |
| >        | Greater than             | x > y   |
| <        | Less than                | x < y   |
| >=       | Greater than or equal to | x >= y  |
| <=       | Less than or equal to    | x <= y  |

# Logical Operators

---

- Used to used to combine conditional statements
- Returns Boolean value

| Operator | Description                                             | Example                                  |
|----------|---------------------------------------------------------|------------------------------------------|
| and      | Returns True if both statements are true                | <code>x &lt; 5 and x &lt; 10</code>      |
| or       | Returns True if one of the statements is true           | <code>x &lt; 5 or x &lt; 4</code>        |
| not      | Reverse the result, returns False if the result is true | <code>not(x &lt; 5 and x &lt; 10)</code> |

# Shorthand Assignment Operators

- Used to assign values to variables
- ++ and -- operators are not available in Python

| Operator | Example | Same As    |
|----------|---------|------------|
| =        | x = 5   | x = 5      |
| +=       | x += 3  | x = x + 3  |
| -=       | x -= 3  | x = x - 3  |
| *=       | x *= 3  | x = x * 3  |
| /=       | x /= 3  | x = x / 3  |
| %=       | x %= 3  | x = x % 3  |
| //=      | x //= 3 | x = x // 3 |
| **=      | x **= 3 | x = x ** 3 |

# Control Flow

---

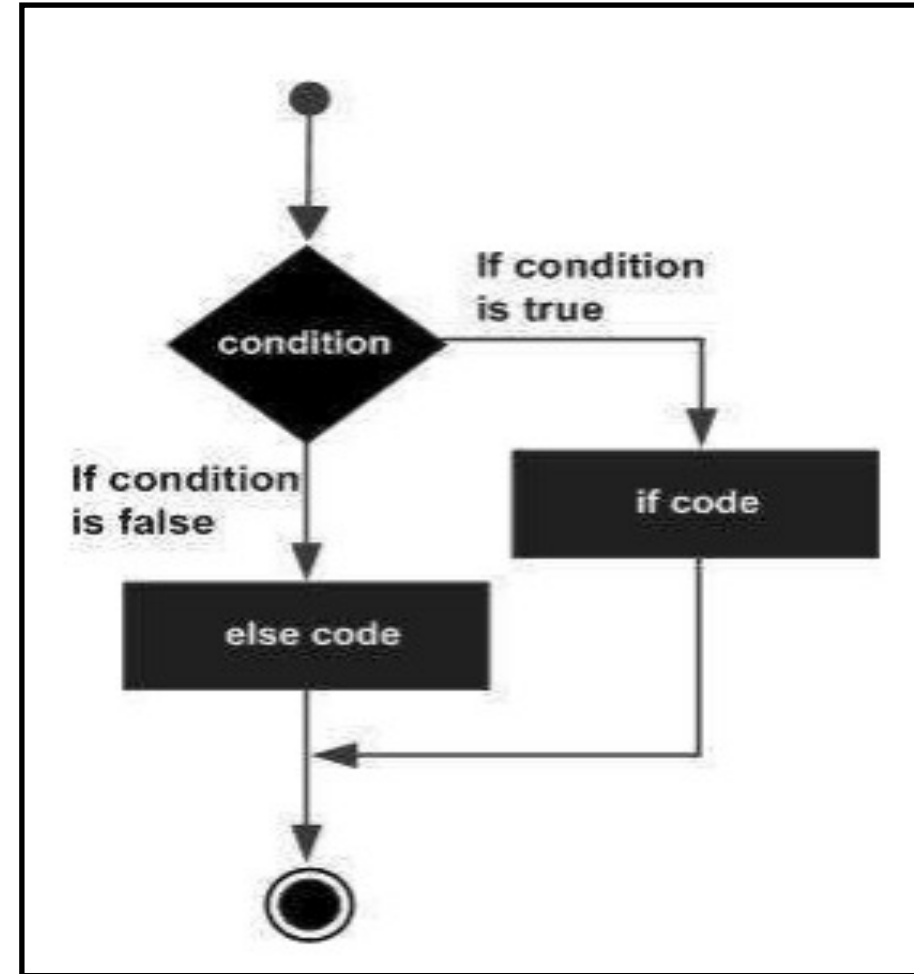
- if
- if-else
- if-elif-else
- for loop
- while loop

# Conditional Statements

```
if boolean-expr :
 S1
S2
```

```
if boolean-expr :
 S1
else:
 S2
S3
```

```
if boolean-expr :
 S1
elif boolean-expr
:
 S2
else:
 S3
S4
```



# Conditional Statements

---

```
In [1]: var = 15
 if var == 15:
 print("Condition is True")
```

Condition is True

```
In [3]: var = 15
 if var == 15:
 print("Condition is True")
```

```
File "<ipython-input-3-fa7c02efd816>", line 2
 if var == 15:
 ^
```

IndentationError: unexpected indent

# Conditional Statements

---

```
In [6]: age = int(input("Enter your age: "))
 if age >= 18:
 print("You are eligible to vote")
 else:
 print("You are not eligible to vote")
```

```
Enter your age: 42
You are eligible to vote
```

```
In [15]: age = int(input("Enter your age: "))
 nationality = input("Enter your nationality: ")
 if age >= 18 and nationality == "INDIAN":
 print("You are eligible to vote")
 else:
 print("You are not eligible to vote")
```

```
Enter your age: 21
Enter your nationality: INDIAN
You are eligible to vote
```

# Conditional Statements

---

```
In [8]: marks = int(input("Enter your marks: "))

if marks >= 85:
 print("Grade O")
elif marks < 85 and marks >= 75:
 print("Grade A+")
elif 65 <= marks < 75:
 print("Grade A")
elif marks < 65 and marks >= 55:
 print("Grade B")
elif 40 <= marks < 55:
 print("Grade C")
else:
 print("Fail")
```

```
Enter your marks: 65
Grade A
```

# Conditional Statements (Nested if, if-else)

```
In [7]: num_1 = int(input("Enter 1st number: "))
num_2 = int(input("Enter 2nd number: "))
num_3 = int(input("Enter 3rd number: "))
if num_1 <= num_2:
 if num_1 <= num_3:
 print("{} is the smallest number".format(num_1))
elif num_2 <= num_3:
 print("{} is the smallest number".format(num_2))
else:
 print("{} is the smallest number".format(num_3))
```

```
Enter 1st number: 34
Enter 2nd number: 12
Enter 3rd number: 78
12 is the smallest number
```

# range function

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

## Syntax

```
range(start, stop, step)
```

```
In [27]: range(0, 10)
```

```
Out[27]: range(0, 10)
```

```
In [28]: range(10)
```

```
Out[28]: range(0, 10)
```

## Parameter Values

| Parameter    | Description                                                                      |
|--------------|----------------------------------------------------------------------------------|
| <i>start</i> | Optional. An integer number specifying at which position to start. Default is 0  |
| <i>stop</i>  | Required. An integer number specifying at which position to stop (not included). |
| <i>step</i>  | Optional. An integer number specifying the incrementation. Default is 1          |

# range function

---

```
In [27]: range(0, 10)
```

```
Out[27]: range(0, 10)
```

```
In [28]: range(10)
```

```
Out[28]: range(0, 10)
```

```
In [31]: range(1, 10)
```

```
Out[31]: range(1, 10)
```

```
In [29]: range(1, 10, 1)
```

```
Out[29]: range(1, 10)
```

```
In [30]: range(1, 10, 3)
```

```
Out[30]: range(1, 10, 3)
```

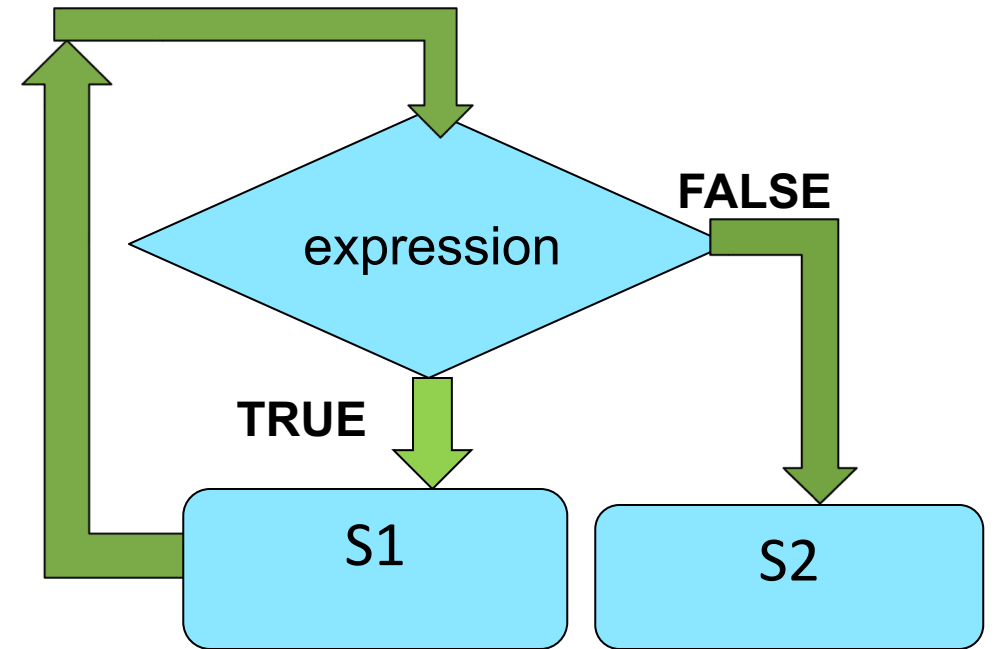
# for Loop

**for variable in sequence:**

S1

S2

1. Check Variable in Sequence
2. If TRUE (Present) then
  - a) execute statement1
  - b) goto step 1.
3. If FALSE (not present) then execute statement2.



# for Loop

```
In [1]: for i in range(0, 10):
 print(i)
```

0  
1  
2  
3  
4  
5  
6  
7  
8  
9

```
In [2]: for i in range(10):
 print(i)
```

0  
1  
2  
3  
4  
5  
6  
7  
8  
9

```
In [3]: for i in range(1, 10):
 print(i)
```

1  
2  
3  
4  
5  
6  
7  
8  
9

```
In [6]: for i in range(1, 10, 2):
 print(i)
```

1  
3  
5  
7  
9

# for Loop (decrementing the value of iterating variable

- The following for loop will not execute as the starting value of iterating variable,  $i = 10$ . The condition checked will be  $i < 0$  (value of 2<sup>nd</sup> parameter), which will return “False”.
- To resolve this problem, we can specify the value for third parameter of range function as a negative value, which means that the iterating variable need to be decremented by the specified value in every iteration

```
In [4]: for i in range(10, 0):
 print(i)
```

```
In [5]: for i in range(10, 0, -1)
 print(i)
```

```
10
9
8
7
6
5
4
3
2
1
```

# for Loop

In [23]: *# Print table of a number*

```
num = int(input("Enter a number: "))
for i in range(1, 11):
 print("{} X {} = {}".format(num, i, num*i))|
```

Enter a number: 3

```
3 X 1 = 3
3 X 2 = 6
3 X 3 = 9
3 X 4 = 12
3 X 5 = 15
3 X 6 = 18
3 X 7 = 21
3 X 8 = 24
3 X 9 = 27
3 X 10 = 30
```

In [7]: `text = "Python"`  
`for ch in text:`  
 `print(ch)`

P  
y  
t  
h  
o  
n

# for Loop

---

```
In [8]: # Factorial of a number
5! = 5*4*3*2*1 = 120
num = int(input("Enter a number: "))
factorial = 1

for i in range(1, num+1):
 factorial = factorial * i

print("Factorial of {} =".format(num), factorial)
```

```
Enter a number: 5
Factorial of 5 = 120
```

# for Loop

```
In [26]: # Fibonacci Sequence / Fibonacci Series
0 1 1 2 3 5 8 13
```

```
count = int(input("How many values you want to print: "))
num_1 = 0
num_2 = 1
print(num_1, "\t", num_2, end="\t")
|
for i in range(3, count+1):
 num_3 = num_1 + num_2
 print(num_3, end="\t")
 num_1 = num_2
 num_2 = num_3
```

How many values you want to print: 10

0            1            1            2            3            5            8            13            21            34

# for Loop

```
In [8]: # Calculate the sum of first 10 natural numbers
sum = 0
for i in range(1, 11):
 sum = sum + i
print("Sum = ", sum)
```

Sum = 55

```
In [16]: # Calculate the sum of odd numbers between 20 - 60
sum = 0
for number in range(20, 61):
 if number % 2 != 0:
 sum = sum + number
print("Sum = ", sum)
```

Sum = 800

# for Loop

```
In [10]: # Factorial of a number
5! = 5 * 4 * 3 * 2 * 1 = 120
num = int(input("Enter a number to calculate it's factorial: "))
fact = 1
for i in range(1, num+1):
 fact = fact * i
print("factorial of {} = {}".format(num, fact))
```

Enter a number to calculate it's factorial: 5  
factorial of 5 = 120

```
In [15]: # Check whether a number is Prime or not
num = int(input("Enter a number: "))
check = False
for i in range(2, num//2+1):
 if num % i == 0:
 check = True
 break

if check == False:
 print(num, " is a prime number")
else:
 print(num, " is not a prime number")
```

Enter a number: 2  
2 is a prime number

# Nested for Loop

---

In [28]:

```

```

```
*
```

```
* *
```

```
* * *
```

```
* * * *
```

```
* * * * *
```

```

```

```
for i in range(1, 6):
 for j in range(1, i+1):
 print("*", end=" ")
 print()
```

```
*
```

```
* *
```

```
* * *
```

```
* * * *
```

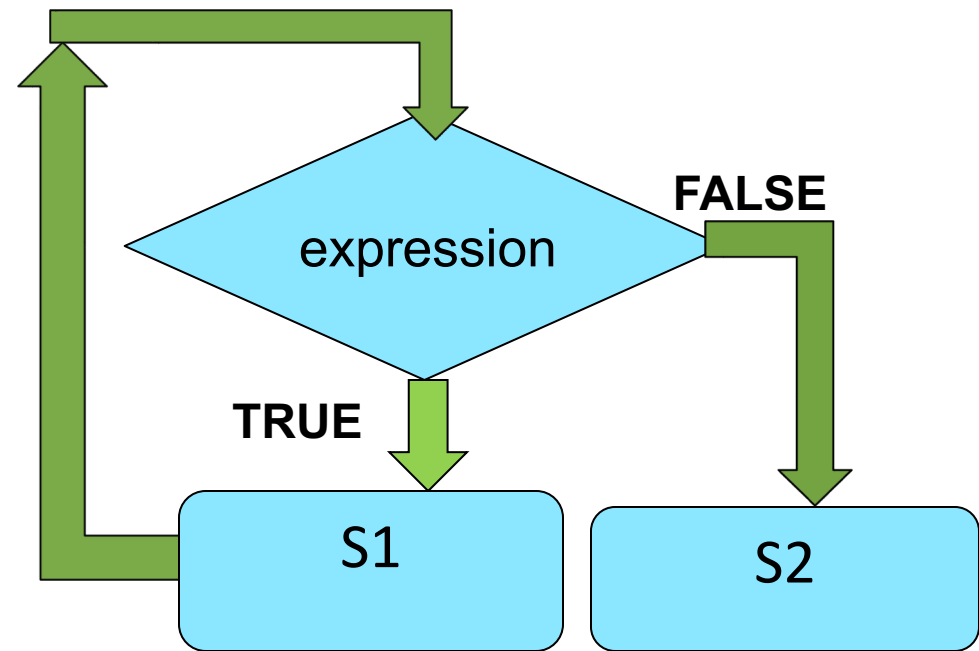
```
* * * * *
```

# while Loop

---

```
while (expression):
 S1
S2
```

1. Evaluate expression
2. If TRUE then
  - a) execute statement1
  - b) goto step 1.
3. If FALSE then execute statement2.



# while Loop

- for loop is used when we know in advance how many times the loop should execute
- while loop is used when the number of iterations are not known in advance

In [163]: *# Calculate the sum of odd numbers between 20 - 60*

*#for loop*

```
sum = 0
for number in range(20, 61):
 if number % 2 != 0:
 sum = sum + number
print("Sum = ", sum)
```

Sum = 800

In [164]: *# Calculate the sum of odd numbers between 20 - 60*

*# while loop*

```
number = 20 # initialization of the iterating variable
sum = 0
while number < 61: # Condition / Expression
 if number % 2 != 0:
 sum = sum + number
 number += 1 # Changing the value of iterating variable

print("Sum = ", sum)
```

Sum = 800

# while Loop

```
In [169]: print("Which operation you want to perform:")
opt = input("Choose any one (+, -, *, /, q - for quit)")

while opt != 'q':
 value1 = int(input("Enter a number: "))
 value2 = int(input("Enter another number: "))
 if opt == '+':
 print("Sum = ", (value1+value2))
 elif opt == '-':
 print("Difference = ", (value1-value2))
 elif opt == '*':
 print("Product = ", (value1*value2))
 elif opt == '/':
 print("Division = ", (value1/value2))
 else:
 print("Invalid option")
 break

print("Which operation you want to perform:")
opt = input("Choose any one (+, -, *, /, q - for quit)")
```

## OUTPUT

```
Which operation you want to perform:
Choose any one (+, -, *, /, q - for quit)+
Enter a number: 12
Enter another number: 34
Sum = 46
Which operation you want to perform:
Choose any one (+, -, *, /, q - for quit)*
Enter a number: 23
Enter another number: 5
Product = 115
Which operation you want to perform:
Choose any one (+, -, *, /, q - for quit)q
```

# for loop/ while Loop

| For loop                                                                                                                                                               | While loop                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| For loop is used to iterate over a sequence of items.                                                                                                                  | While loop is used to repeatedly execute a block of statements while a condition is true.                                                            |
| For loops are designed for iterating over a sequence of items. Eg. list, tuple, etc.                                                                                   | While loop is used when the number of iterations is not known in advance or when we want to repeat a block of code until a certain condition is met. |
| For loop require a sequence to iterate over.                                                                                                                           | While the loop requires an initial condition that is tested at the beginning of the loop.                                                            |
| For loop is typically used for iterating over a fixed sequence of items                                                                                                | While loop is used for more complex control flow situations.                                                                                         |
| For loop is more efficient than a while loop when iterating over sequences, since the number of iterations is predetermined and the loop can be optimized accordingly. | While loop may be more efficient in certain situations where the condition being tested can be evaluated quickly.                                    |
| Every for loop can be converted into while loop                                                                                                                        | Every while loop can not be converted into for loop                                                                                                  |

# Finite & Infinite Loop

---

- Finite Loop:
  - A loop that run a block of code a certain number of times.
- Infinite Loop:
  - A loop that run a block of code an unspecified number of times (infinite times).
  - Ex:

```
i = 0
while True:
 print(i)
 i += 1
```

```
i = 5
while i < 10:
 print(i)
 i -= 1
```

```
i = 5
while i < 10:
 print(i)
```

# break and continue statements

---

- break:
  - used to stop the execution of the loop
  - used to break out a for loop, or a while loop.
- continue:
  - used to skip remaining statements of current iteration of the loop
  - used to end the current iteration in a for loop or a while loop and continues to the next iteration.
  - Make sure continue does not bypass update-expression for while loops

# pass statement

---

- Empty code is not allowed in loops, function definitions, class definitions, or in if statements.
- The pass statement is used as a placeholder for future code.
- When the pass statement is executed, nothing happens, but you avoid getting an error when empty code is not allowed.

# break, continue and pass statements

```
In [23]: # Print number 1 – 10
Number 5 should not be printed
loop should stop when number is equals to 8

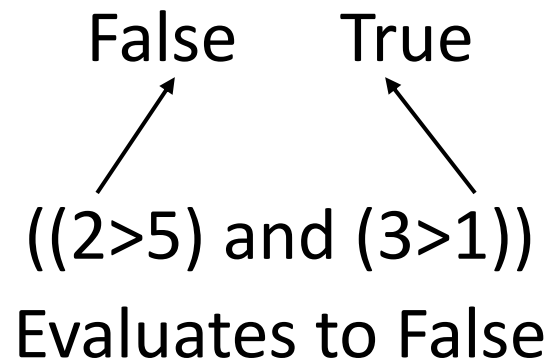
for number in range(1, 11):
 if number == 5:
 continue
 if number == 8:
 print(number)
 break
 if number == 6:
 pass
 print(number)
```

```
1
2
3
4
6
7
8
```

# Short circuit (lazy evaluation)

---

- Do not evaluate the second operand of binary short-circuit logical operator if the result can be deduced from the first operand
  - Also applies to nested logical operators
- Ex: In the below expression, because two conditions are combined with “and” operator and The “and” operator returns True only if both conditions are True. because the first part (2>5) returns false, there is no need to evaluate the 2<sup>nd</sup> part (3>1). Because the final result will be false only.



# String Manipulations

---

- Subscript operator – [] is subscript operator
- Indexing
- slicing a string
- converting strings to numbers and vice versa
- split function

# Strings

---

- Strings in Python have type `str`
- They represent sequence of characters
  - Python does not have a type corresponding to character.
- Strings are enclosed in single quotes(') or double quotes("")
  - Both are equivalent
- Backslash (\) is used to escape quotes and special characters

# Concatenate & Repeat a String

---

- In Python, **+** and **\*** operations have special meaning when operating on strings
  - **+** is used for concatenation of (two) strings
  - **\*** is used to repeat a string, an **int** number of time
- Function/Operator Overloading

```
str1 = "Python"
str2 = "Programming"
name = str1 + str2
print(name)
name = str1 + " " + str2
print(name)
print(name*3)
```

PythonProgramming

Python Programming

Python ProgrammingPython ProgrammingPython Programming

# Length of a String

---

- len() function is used to calculate the length of a string.

```
In [3]: str1 = "Python Programming"
 length = len(str1)
 print(length)
```

18

- `\n` is a **single** character

```
In [4]: str1 = "Python\nProgramming"
 length = len(str1)
 print(str1)
 print(length)
```

Python  
Programming  
18

# Indexing in a String

- Strings are indexed
- First character of a string has index 0
- Last character of a string has index (length-1)
- Indexing of String
  - Positive Indexing
    - Starts from 0 and moves from left to right
    - Indexing starts counting from left / start of the string
    - Index 0 – First Character, Index 1 – Second Character and so on
  - Negative Indexing
    - Starts from -1 and moves from right to left
    - Indexing starts counting from right / end of the string
    - Index -1 – last Character, Index -2 – Second last Character and so on
  - Using an index that is too large or too small results in “**index out of range**” error

```
In [8]: text = "Python Programming"
 print(text)
```

Python Programming

```
In [9]: print(text[5])
```

n

```
In [73]: # Negative Indexing
 text = "Python Programming"
 print(text[-1])
```

g

```
In [74]: print(text[-2])
```

n

# Understanding Indexing in a String

---

| Positive Indexing | P  | y  | t  | h  | o  | n  | Negative Indexing |
|-------------------|----|----|----|----|----|----|-------------------|
|                   | 0  | 1  | 2  | 3  | 4  | 5  |                   |
|                   | -6 | -5 | -4 | -3 | -2 | -1 |                   |

# Indexing in a String

- Using an index that is too large or too small results in “**index out of range**” error.

```
In [75]: text = "Python Programming"
print(text[-20])
```

```

IndexError Traceback (most recent call last)
<ipython-input-75-0ab757140b93> in <module>
----> 1 print(text[-20])

IndexError: string index out of range
```

```
In [76]: text = "Python Programming"
print(text[20])
```

```

IndexError Traceback (most recent call last)
<ipython-input-76-facfc62f81b5> in <module>
 1 text = "Python Programming"
----> 2 print(text[20])

IndexError: string index out of range
```

# Accessing Strings through Loops

- Accessing by Index number

```
In [10]: text = "Python Programming"
length = len(text)
print("length = ", length)
for i in range(length):
 print(text[i])
```

```
length = 18
```

```
P
y
t
h
o
n

P
r
o
g
r
a
m
m
i
n
g
```

- Accessing by sequence

```
In [11]: text = "Python Programming"
for i in text:
 print(i)
```

```
P
y
t
h
o
n

P
r
o
g
r
a
m
m
i
n
g
```

# Slicing a String

---

- To obtain a substring
- `s[start:end:step]` means substring of `s` starting at index `start` and ending at index `end-1`, `step` is the incrementation in the index number to access next value.
- `start`, `end` and `step` all are optional
  - If `start` is omitted, it defaults to 0
  - If `end` is omitted, it defaults to the length of string
  - If `step` is omitted, it defaults to 1
- `s[:]` is same as `s[0:len(s)]`, that is same as `s`

# Slicing a String

|     |     |     |     |     |     |     |     |     |    |    |    |    |    |    |    |    |    |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|----|----|----|----|----|----|----|----|
| P   | y   | t   | h   | o   | n   |     | P   | r   | o  | g  | r  | a  | m  | m  | i  | n  | g  |
| 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
| -18 | -17 | -16 | -15 | -14 | -13 | -12 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

In [22]: *# String Slicing*

```
text = "Python Programming"
slice = text[2:10]
print(slice)
print(text[2:10])
```

thon Pro  
thon Pro

In [25]: `print(text[2:])`  
`print(text[:10])`

thon Programming  
Python Pro

In [78]: `print(text[:])`

Python Programming

In [80]: `print(text[2:13:2])`

to rga

In [82]: *# Negative Indexing*

```
text = "Python Programming"
print(text[-1])
print(text[-2])
```

g  
n

In [87]: `print(text[1:-1])`

ython Programmin

In [30]: `print(text[-3:-1])`

in

In [86]: `print(text[-10:-1:2])`

rgamn

In [31]: `print(text[-8:0])`

In [32]: `print(text[-8:])`

gramming

# Out of Range Slicing

- Out of range indices are ignored for slicing
- when start and end have the same sign and if start  $\geq$  end, empty slice is returned

```
In [75]: text = "Python Programming"
print(text[-20])
```

```

IndexError Traceback (most recent call last)
<ipython-input-75-0ab757140b93> in <module>
----> 1 print(text[-20])

IndexError: string index out of range
```

```
In [76]: text = "Python Programming"
print(text[20])
```

```

IndexError Traceback (most recent call last)
<ipython-input-76-facfc62f81b5> in <module>
 1 text = "Python Programming"
----> 2 print(text[20])

IndexError: string index out of range
```

# converting strings to numbers and vice versa

- `int()` function is used to convert strings to numbers. Only digits written as strings can be converted in to numbers.
- `float()` function is used to convert strings to decimal numbers. Only digits written as strings can be converted in to decimal numbers.
- `str()` function is used to convert numbers to strings.

```
In [121]: value1 = '12'
 print(type(value1))
 value2 = int(value1)
 print(type(value2))
```

```
<class 'str'>
<class 'int'>
```

```
In [123]: value1 = 12
 print(type(value1))
 value2 = str(value1)
 print(type(value2))
```

```
<class 'int'>
<class 'str'>
```

```
In [122]: value1 = '12.5'
 print(type(value1))
 value2 = float(value1)
 print(type(value2))|
```

```
<class 'str'>
<class 'float'>
```

# converting strings to numbers and vice versa

---

```
In [124]: value1 = 'Python'
 print(type(value1))
 value2 = int(value1)
 print(type(value2))
```

```
<class 'str'>
```

---

**ValueError** Traceback (most recent call last)

**<ipython-input-124-3988b9ee06a0>** in **<module>**

```
 1 value1 = 'Python'
 2 print(type(value1))
----> 3 value2 = int(value1)
 4 print(type(value2))
```

**ValueError:** invalid literal for int() with base 10: 'Python'

# String Methods / Functions

## 1. **lower()** –

- i. Converts string into lower case
- ii. Syntax – string.lower()

## 2. **upper()** –

- i. Converts string into upper case
- ii. Syntax – string.upper()

## 3. **casefold()** -

- i. Converts string into lower case
- ii. Syntax – string.casefold()

## 4. **swapcase()** –

- i. Converts lowercase letters of the string into uppercase and uppercase into lowercase
- ii. Syntax – string.swapcase()

## 5. **title()** –

- i. Converts first letter of each word of string into upper case and remaining letters into lowercase
- ii. Syntax – string.title()

## 6. **capitalize()** –

- i. Converts first letter of string into upper case and remaining letters into lowercase
- ii. Syntax – string.capitalize()

```
In [12]: text = "Python Programming"
 print(text.lower())

python programming
```

```
In [13]: print(text.upper())

PYTHON PROGRAMMING
```

```
In [15]: print(text.casefold())

python programming
```

```
In [16]: print(text.swapcase())

pYTHON pROGRAMMING
```

```
In [18]: text = "Python program"
 print(text.title())

Python Program
```

```
In [20]: text = "Python Programming"
 print(text.capitalize())

Python programming
```

```
In [21]: text = "123@Python Programming"
 print(text.capitalize())

123@python programming
```

# casefold() / lower() String Methods

casefold() method is similar to the lower() method, but the casefold() method is stronger, more aggressive, meaning that it will convert more characters into lower case, and will find more matches when comparing two strings and both are converted using the casefold() method.

In [3]:

```
Define a string
string1 = "ß" # A German Letter
print("Original string")
print(string1)

Store the output of casefold() method in string2
print("Convert using casefold()")
string2 = string1.casefold()
print(string2)

Store the output of lower() method in string3
print("Convert using lower()")
string3 = string1.lower()
print(string3)
```

Original string

ß

Convert using casefold()

ss

Convert using lower()

ß

| lower() Method                                            | casefold() Method                                                                                   |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| It converts only ASCII characters to lowercase.           | It converts both ASCII and non-ASCII characters to lowercase.                                       |
| It is suitable for only case-sensitive string comparison. | It is suitable for both case-sensitive and case-insensitive string comparisons.                     |
| It provides high performance than casefold().             | It works slower than lower().                                                                       |
| Works well for English language text                      | Works well for English and other languages as well e.g., German (Example given in the code snippet) |

# String Functions (count())

**count(x)** – returns the number of times a specified value (x) appears in the string.

**1. Syntax:** string.count(value, start, end)

- i. value - Required. A String. The string to value to search for
- ii. start - Optional. An Integer. The position to start the search. Default is 0
- iii. end - Optional. An Integer. The position to end the search. Default is the end of the string

**2. Case Insensitive count in a string -**

- a) **string.upper().count(x)** – First convert the string into upper case and then count the letter 'x'
- b) **string.lower().count(x)** – First convert the string into lower case and then count the letter 'x'

```
In [1]: text = "python programming class"
print("count of letter 'a' = ",text.count("a"))
print("count of letter 'a' from index no 2 to 13 = ",text.count("a", 2, 14))
print("count of letter 'on' = ",text.count("on"))
```

```
count of letter 'a' = 2
count of letter 'a' from index no 2 to 13 = 1
count of letter 'on' = 1
```

```
In [2]: text = "Python programming class"
print("count of letter 'p' = ",text.count("p"))
print("count of letter 'p' or 'P' = ",text.lower().count("p"))
```

```
count of letter 'p' = 1
count of letter 'p' or 'P' = 2
```

# String Functions (index())

**index(x)** –

1. finds the first occurrence of the specified value (x).
2. **The index() method raises an exception if the value is not found.**
3. Return the index number (indices) of the first occurrence of letter 'x' in a string
4. Syntax – string.index(value, start, end)
  - i. value - Required. The value to search for
  - ii. start - Optional. Where to start the search. Default is 0
  - iii. end - Optional. Where to end the search. Default is to the end of the string
5. **The index() method is almost the same as the find() method, the only difference is that the find() method returns -1 if the value is not found.**
6. **Case Insensitive search for Index number in a string 'text' -**
  - a) **text.upper().index(x)** – First convert the string into upper case and then count the letter 'x'
  - b) **text.lower().index(x)** – First convert the string into lower case and then count the letter 'x'

```
In [5]: # index() method
text = "Python programming class"
print("Index of letter 'p' = ",text.index("p"))
print("Index of letter 'p' from index no 2 to 13 = ",text.index("p", 2, 14))
print("Index of letter 'on' = ",text.index("on"))
print("Index of letter 'p'/'P' = ",text.lower().index("p"))
```

```
Index of letter 'p' = 7
Index of letter 'p' from index no 2 to 13 = 7
Index of letter 'on' = 4
Index of letter 'p'/'P' = 0
```

# String Functions (find())

**find(x)** –

1. finds the first occurrence of the specified value (x).
- 2. The find() method returns -1 if the value is not found**
3. Return the index number (indices) of the first occurrence of letter 'x' in a string
4. Syntax – string.find(value, start, end)
  - i. value - Required. The value to search for
  - ii. start - Optional. Where to start the search. Default is 0
  - iii. end - Optional. Where to end the search. Default is to the end of the string
- 5. The index() method is almost the same as the find() method, the only difference is that the find() method returns -1 if the value is not found.**
- 6. Case Insensitive search for Index number in a string 'text' -**
  - a) text.upper().find(x)** – First convert the string into upper case and then count the letter 'x'
  - b) text.lower().find(x)** – First convert the string into lower case and then count the letter 'x'

```
In [6]: # find() method
text = "Python programming class"
print("Index of letter 'p' = ",text.find("p"))
print("Index of letter 'p' from index no 2 to 13 = ",text.find("p", 2, 14))
print("Index of letter 'on' = ",text.find("on"))
print("Index of letter 'p'/ 'P' = ",text.lower().find("p"))
```

```
Index of letter 'p' = 7
Index of letter 'p' from index no 2 to 13 = 7
Index of letter 'on' = 4
Index of letter 'p'/ 'P' = 0
```

# String Functions (index() / find())

The index() method is almost the same as the find() method, the only difference is that the find() method returns -1 if the value is not found.

```
In [8]: text = "Python programming class"
print("Index of letter 'Java' = ",text.find("Java"))
```

```
Index of letter 'Java' = -1
```

```
In [9]: text = "Python programming class"
print("Index of letter 'p' = ",text.index("Java"))
```

```

ValueError Traceback (most recent call last)
```

```
<ipython-input-9-6c8fd50fca2d> in <module>
```

```
1 text = "Python programming class"
```

```
----> 2 print("Index of letter 'p' = ",text.index("Java"))
```

```
ValueError: substring not found
```

**rfind() and rindex()** methods -

Return the index number (indices) of the last occurrence of the specifies value.

```
In [43]: text = "Python Programming"
print(text.rfind("o"))
print(text.rindex("o"))
```

```
9
9
```

# String Methods (strip())

## strip() method -

1. Returns a trimmed version of the string
2. Removes any leading (at the beginning of the string), and trailing characters/whitespaces (at the end of the string).
3. Syntax: **string.strip(characters)**
  - i. characters - Optional. A set of characters to remove as leading/trailing characters
  - ii. if no character is passed inside the method, whitespaces will be removed.

**lstrip()** - Removes any leading (at the beginning of the string) characters/whitespaces.

**rstrip()** - Removes any trailing (at the end of the string) characters/whitespaces.

```
In [160]: text = "@Python Programming@"
print(text.strip('@'))
```

Python Programming

```
In [158]: text = " Python Programming "
print(text)
print(text.strip())
```

Python Programming  
Python Programming

```
In [10]: text = "Python Programming P"
print(text.strip("P"))
```

ython Programming

```
In [11]: text = "Python Programming P"
print(text.strip("Py"))
```

thon Programming

```
In [12]: text = "Python Programming P"
print(text.lstrip("P"))
```

ython Programming P

```
In [13]: text = "Python Programming P"
print(text.rstrip("P"))
```

Python Programming